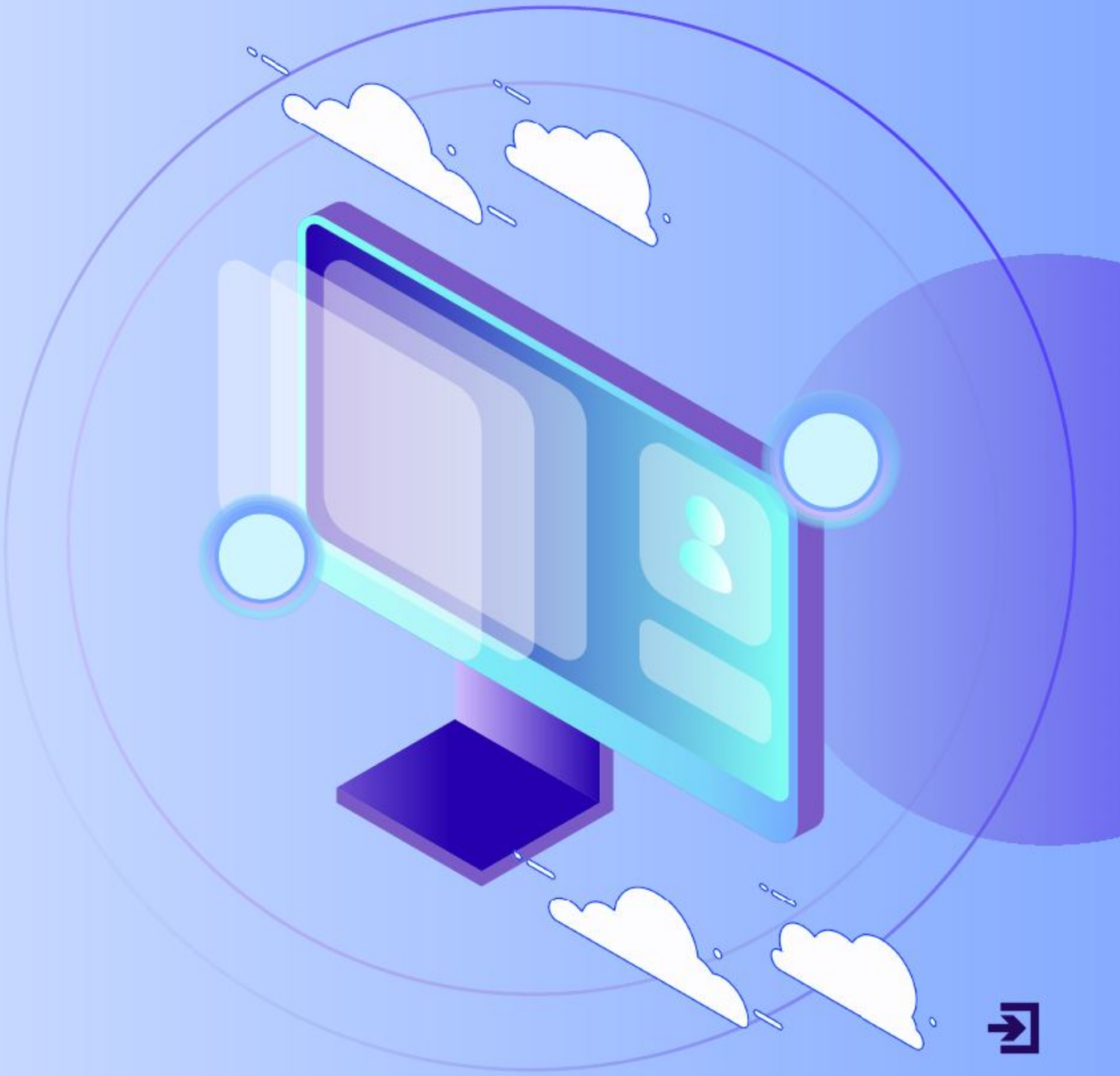
Cyber2

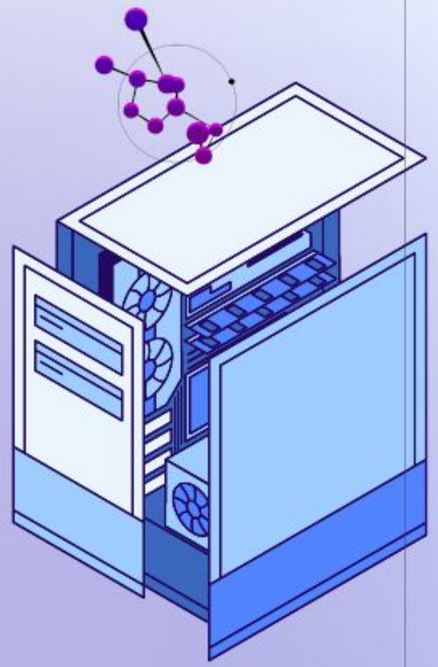
SYMFONY TYPE

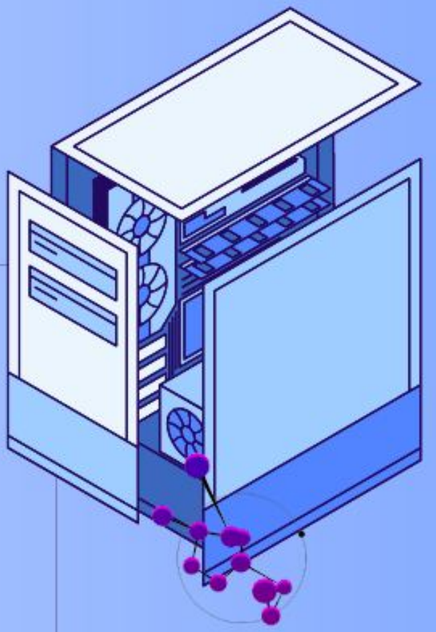
Gestion des accès • Symfony 7.4



SymfonyType

NOS OBJECTIFS





- Sécuriser l’inscription et la connexion
- Protéger les mots de passe
- Administrer les comptes et les accès
- Créer une base réutilisable sur GitHub

SymfonyType

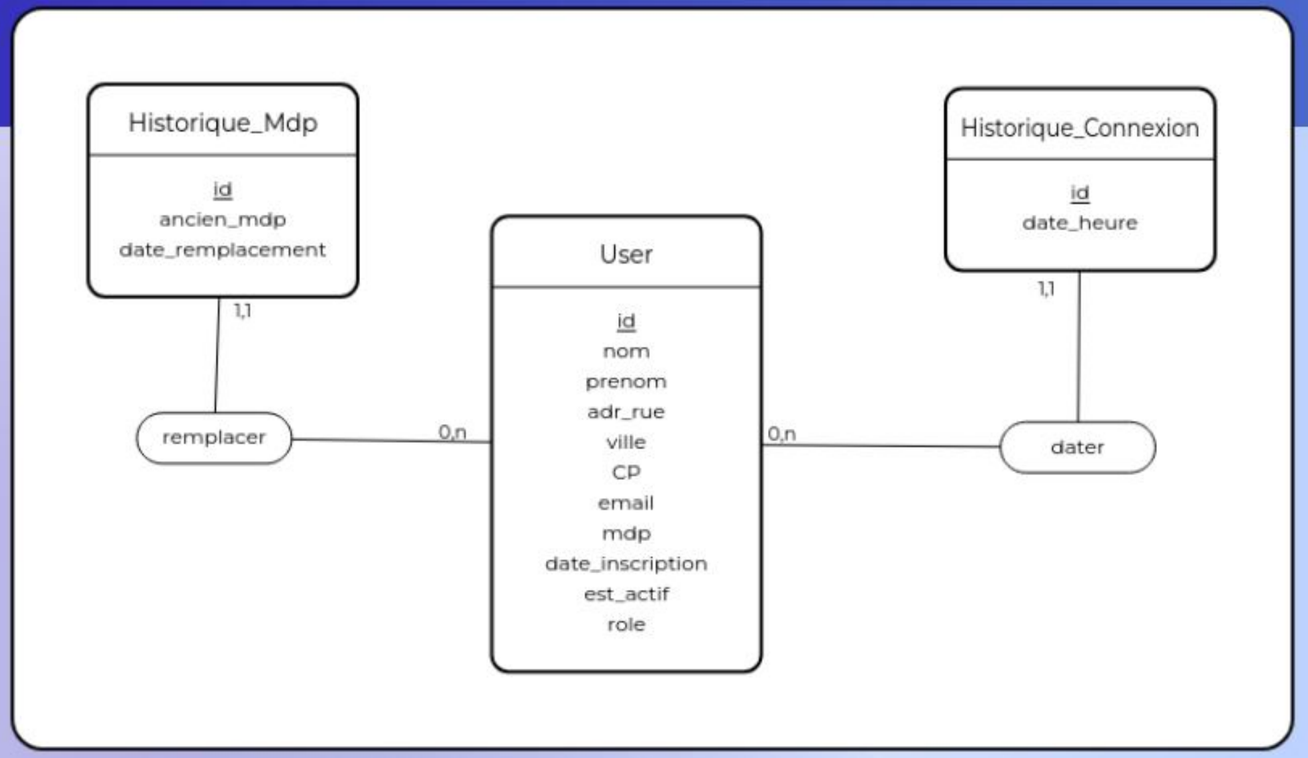
NOTRE ENVIRONNEMENT

Symfony 7.4
Apache : serveur web • MariaDB : base

4719 / 4743 : machines de travail
GitHub : dépôt de code partagé
5183 : site commun à l'équipe

SymfonyType

MODÈLE DE DONNÉES



User : identité, rôles et mot de passe haché

Historique_Mdp : anciens mots de passe hachés

Historique_Connexion : dates des connexions réussies

SymfonyType

INSCRIPTION CONTRÔLÉE

- Les coordonnées sont obligatoires.
- L'email doit être valide et unique.
- Symfony vérifie les données saisies.
- Le compte reçoit les droits d'un utilisateur.



BIENVENUE

Créer un compte

Les mots de passe doivent contenir au moins 10 caractères, une majuscule, une minuscule, un chiffre et un caractère spécial. Les mots de passe compromis sont refusés.

Adresse e-mail

Nom

Prénom

Adresse

Ville

Code postal

Mot de passe

Confirmer le mot de passe

Créer mon compte

MOTS DE PASSE

SymfonyType

- Au moins **10 caractères** : majuscule, minuscule, chiffre et symbole.
- Les mots de passe **faibles** ou **compromis** sont refusés.

- Seule une empreinte (**hachage**) est **stockée**.
- Mêmes règles à **l'inscription** et lors d'un **changement** de mot de passe.

src/Security/PasswordPolicy.php

Contraintes Symfony · extraits, messages omis

```
new Assert\Length(min: 10, max: 4096, …),
new Assert\Regex(pattern: '/[a-z]/', …),
new Assert\Regex(pattern: '/[A-Z]/', …),
new Assert\Regex(pattern: '/\d/', …),
new Assert\Regex(pattern: '/[^a-zA-Z0-9]/', …),
new Assert\PasswordStrength(
    minScore: Assert\PasswordStrength::STRENGTH_MEDIUM, …),
new Assert\NotCompromisedPassword(…),
```

SymfonyType

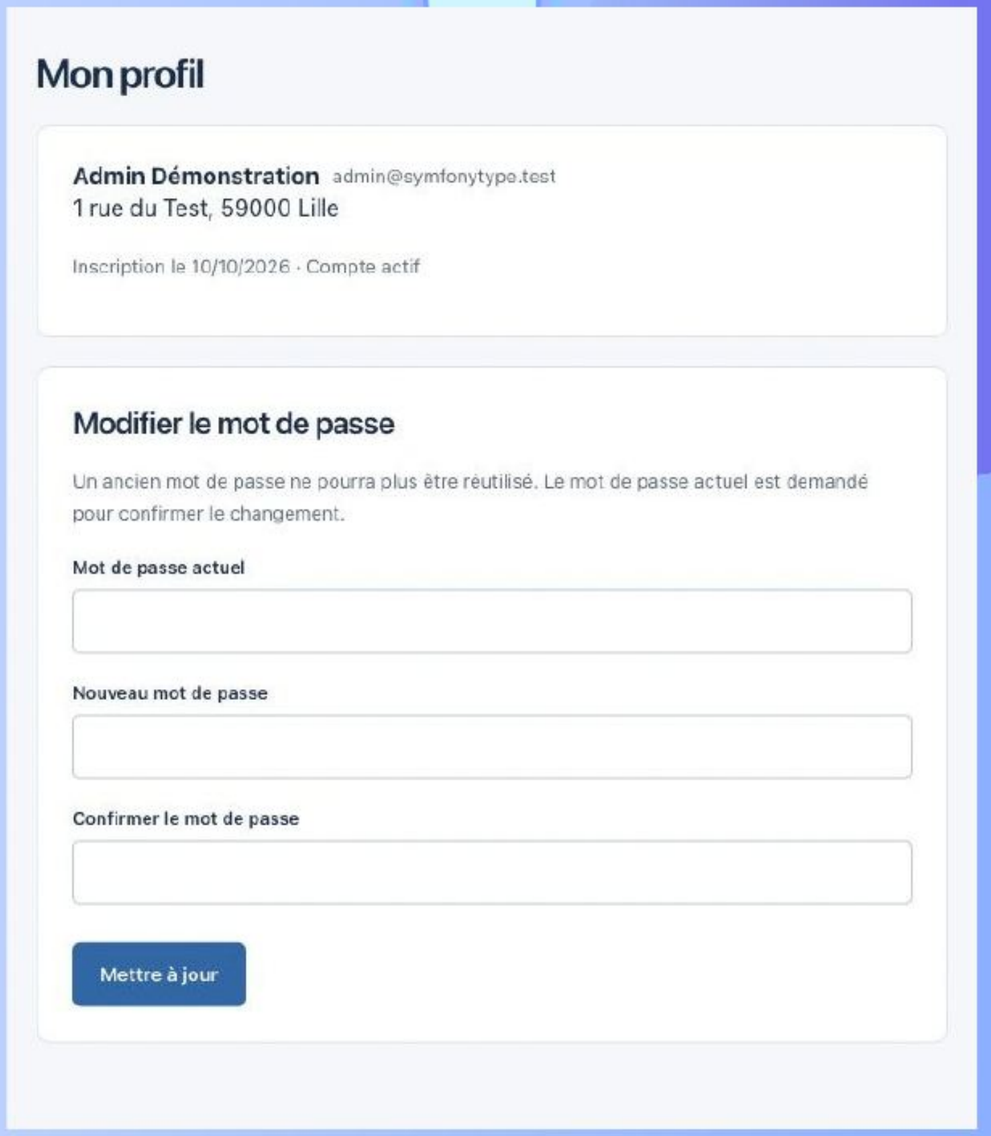
NON-RÉUTILISATION

- L’utilisateur **confirme** son mot de passe actuel.

- Le nouveau est vérifié à partir des empreintes des anciens mots de passe : **aucun ne peut resservir.**

- **L’ancienne empreinte est conservée** à chaque changement.

- Exemple : après A puis B, A reste interdit.



SymfonyType

CONNEXION PROTÉGÉE

config/packages/security.yaml

Extraits · authentification, force brute et accès

```yaml
form_login:
    login_path: app_login
    check_path: app_login
    enable_csrf: true
# …
login_throttling:
    max_attempts: 3
    interval: '15 minutes'
# …
access_control:
    - { path: ^/admin(?:/|$), roles: ROLE_ADMIN }
    - { path: ^/profil(?:/|$), roles: ROLE_USER }
```

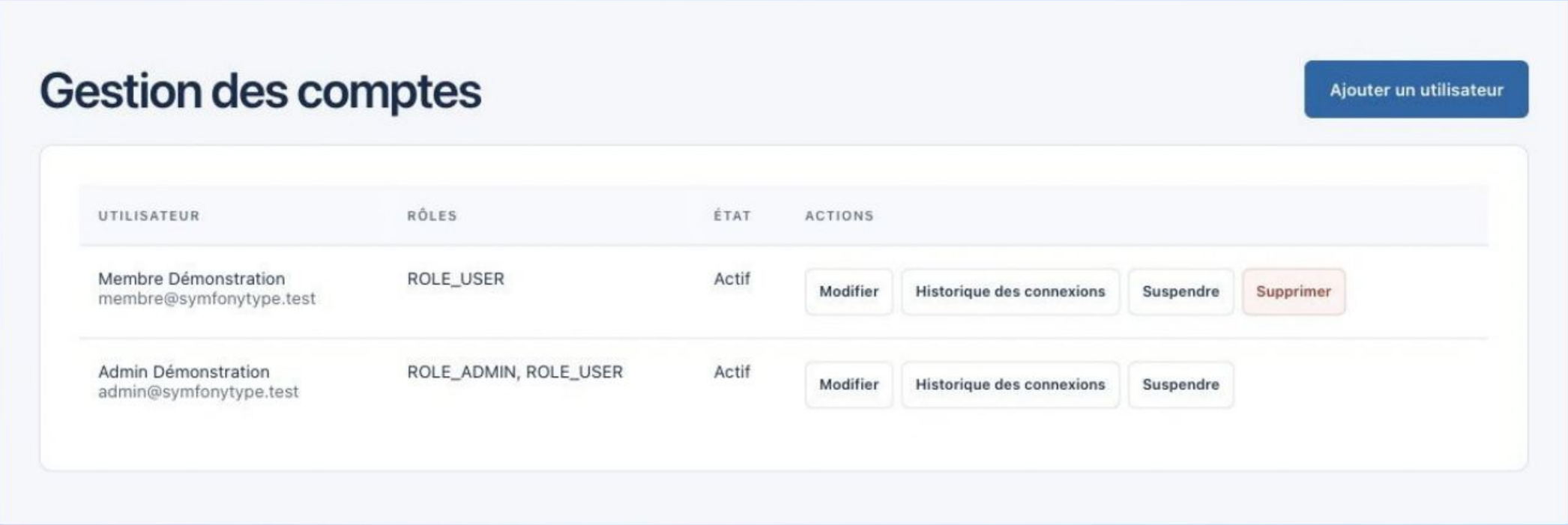
Symfony **vérifie l'email et le mot de passe**.

3 échecs autorisés par période de 1**5 minutes**,
pour un même **identifiant** et une **même IP**.

Un **jeton CSRF** protège les formulaires contre les actions envoyées par un autre site.

SymfonyType

ADMINISTRATION



L'administrateur **crée**, **modifie**, **supprime** ou **suspend** les comptes et **choisit leurs rôles**.
Un compte suspendu perd l'accès dès sa prochaine requête au site.

SymfonyType

HISTORIQUE DES CONNEXIONS

À chaque connexion réussie, Symfony **enregistre la date, l'heure et l'utilisateur.**

Seul l'administrateur peut **consulter cet historique.**

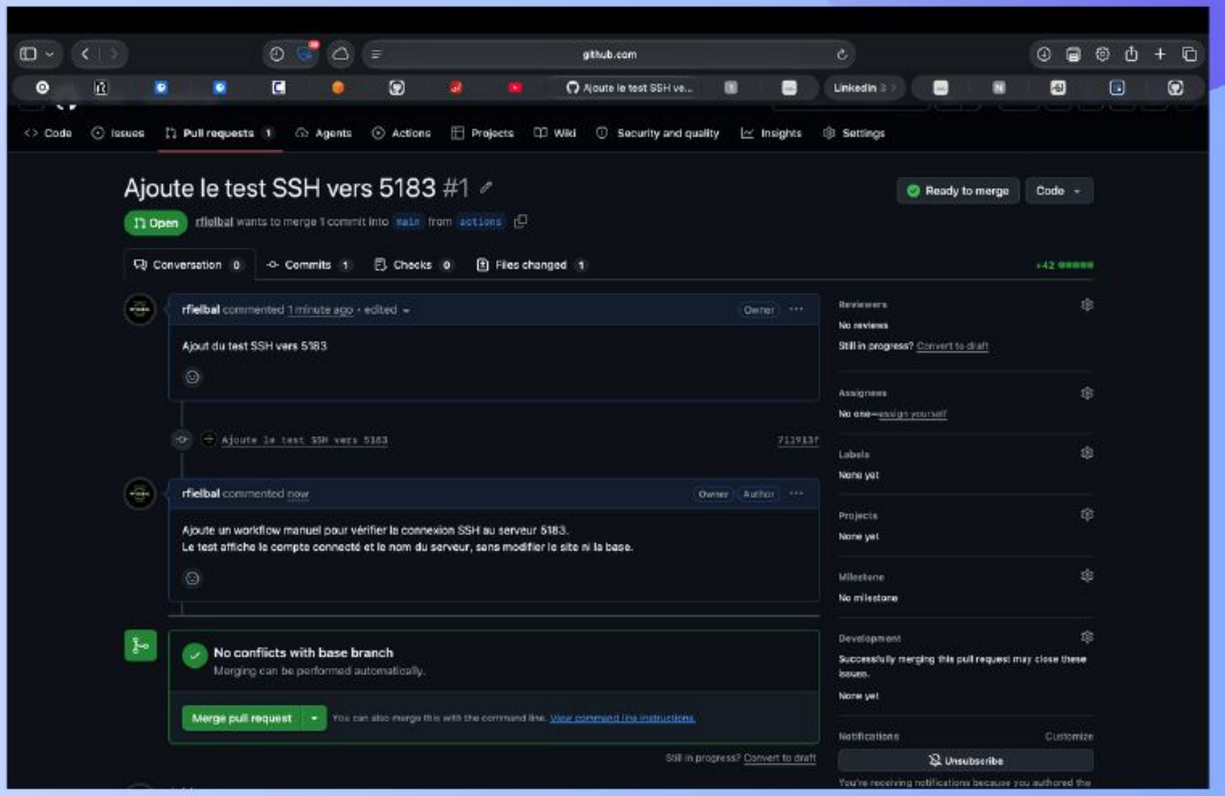


SymfonyType

PULL REQUEST // FAIRE RELIRE LE CODE

1. Une branche isole notre modification.
2. Un commit enregistre le travail.
3. Le push envoie cette branche sur GitHub.
4. La pull request propose son ajout à main.

L'équipier **relit**, **approuve** ou **demande** une **correction**. La fusion **ajoute le code** à main.



SymfonyType

CONFIGURER LA RELECTURE

RÈGLE ACTIVE SUR LA BRANCHE MAIN

Settings → Rules → Rulesets : cibler main et activer la règle.
Require a pull request before merging : passer par une PR.
Required approvals = 1 : l'équipier doit approuver avant fusion.

POUR CHAQUE MODIFICATION

Ouvrir une PR depuis notre branche vers main.
Le choisir dans Reviewers ; il relit dans Files changed.
Review changes → Approve, puis Merge pull request.



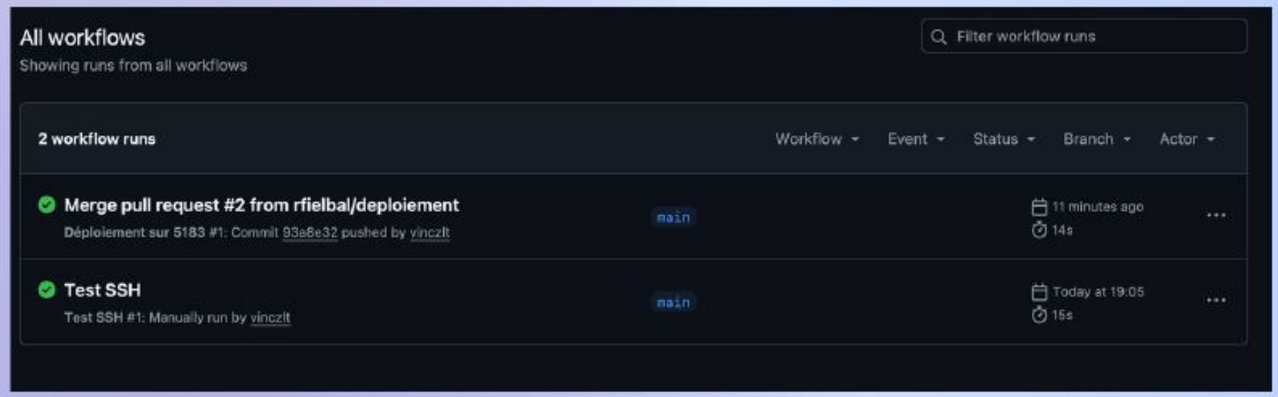
SymfonyType

GITHUB ACTIONS

.github/workflows/deploy.yml

Extraits · déclenchement et commandes sur 5183

```
on:
  push:
    branches: [main]
# … Sur le serveur, après connexion SSH :
git pull --ff-only origin main
composer install --no-interaction --prefer-dist
php bin/console doctrine:migrations:migrate --no-interaction
php bin/console cache:clear
```



APRÈS LA FUSION DANS MAIN

Un push déclenche le fichier deploy.yml. GitHub démarre une machine temporaire (le runner), qui se connecte à 5183 par SSH.

SUR LE SERVEUR 5183

Le code est récupéré, les bibliothèques installées, les migrations appliquées et le cache renouvelé. Le résultat apparaît dans l'onglet Actions.

Ce workflow déploie le site ; les tests restent locaux.

SymfonyType

SETTINGS → SECRETS AND VARIABLES → ACTIONS → NEW REPOSITORY SECRET

- NAME = NOM DU SECRET
- SECRET = SA VALEUR

EXEMPLE : SSH_HOST = S3.NUAGE-PEDA.FR

SECRETS GITHUB

.github/workflows/deploy.yml

Extrait réel · noms des secrets, jamais leurs valeurs

```yaml
env:
  SSH_HOST: ${{ secrets.SSH_HOST }}
  SSH_PORT: ${{ secrets.SSH_PORT }}
  SSH_USER: ${{ secrets.SSH_USER }}
  SSH_PRIVATE_KEY: ${{ secrets.SSH_PRIVATE_KEY }}
  SSH_KNOWN_HOSTS: ${{ secrets.SSH_KNOWN_HOSTS }}
```

SSH_HOST = s3.nuage-peda.fr
Adresse du serveur à contacter.

SSH_PORT = 5183 • **SSH_USER** = login5183
Port SSH et compte utilisés pour la connexion.

SSH_PRIVATE_KEY = clé privée dédiée
Permet la connexion

SSH_KNOWN_HOSTS = identité SSH du serveur
Permet de vérifier que le serveur est le bon.

${{ secrets.SSH_HOST }} lit la valeur enregistrée.